Aprendizaje por Refuerzo aplicado a la Automatización de Pruebas en Videojuegos

Realizado por Maximiliano González Segura - 04/02/2026

Índice:

1. Análisis del sector *3*

2. Identificación del problema real *5*

3. Escenario de Uso Real *6*

4. Impacto y Beneficios Esperados *7*

5. Viabilidad y Oportunidad de Negocio *7*

6. Conclusiones y referencias *8*

1. Análisis del sector

Mi proyecto se sitúa en el sector de los videojuegos, más concretamente en el subsector de **Herramientas de Desarrollo y Aseguramiento de Calidad (QA)** mediante Inteligencia Artificial.

Este sector se caracteriza por una alta competitividad y una necesidad constante de optimizar los tiempos de producción. El uso de IA para automatizar tareas repetitivas, como el playtesting (el cuál sirve para probar niveles dentro de los juegos para detectar errores) se está convirtiendo en una tendencia emergente que busca reducir costes y mejorar la calidad del producto final.

Caracterización de Empresas del Sector

He elegido estas tres empresas porque representan bien el mercado ya operan en este ámbito y serían potenciales clientes o referentes para la solución propuesta:

<u>Electronic Arts (EA) / Ubisoft:</u>

- Gran Corporación Multinacional
- Estructura jerárquica compleja con departamentos masivos dedicados exclusivamente al control de calidad de los juegos. Invierten millones en testers humanos los cuáles podrían ser fácilmente sustituibles por la IA.
- Buscan activamente reducir el coste de las miles de horas de testing manual. Una IA que pudiera jugar sola reduciría drásticamente estos costes.

<u>Unity Technologies:</u>

- Proveedor de Tecnología
- Empresas tecnológicas que desarrollan motores gráficos y herramientas como plugins de IA para venderlos a otros estudios de videojuegos.

- Su modelo es vender herramientas. Mi proyecto, basado en Python y PyTorch, encaja como un prototipo de plugin para automatizar pruebas de jugabilidad.

RobTop Games:

- Estudio Indie
- Estructura muy pequeña y ágil. El desarrollo depende fuertemente de la comunidad y del creador principal.
- Al tener recursos limitados, herramientas automáticas que validen la dificultad de los niveles sin intervención humana serían vitales para mantener la eficiencia.
-

Características Organizativas Comunes

En todas las empresas mencionadas hay figuras como los **desarrolladores de gameplay, diseñadores de niveles, ingenieros de IA y testers de calidad**. Por otra parte, el ciclo de desarrollo incluye fases de "Prueba y Error" iterativas que consumen **hasta el 40% del tiempo de producción**, todo ese tiempo podría ser reinvertido para una mayor eficiencia. En todas hay uso extensivo de motores gráficos, **lenguajes de scripting (C, Python…)** y, cada vez más, librerías de Deep Learning como **PyTorch o TensorFlow** para comportamientos de NPCs (Los NPCs son los personajes que puedes interactuar dentro de los videojuegos pero son creados y gestionados por el propio videojuego).

2. Identificación del problema real

En el desarrollo de videojuegos de plataformas y habilidad, la validación de niveles es un proceso crítico. Actualmente, existe una **ineficiencia significativa en la fase de testing y balanceo de dificultad**. Los desarrolladores deben jugar manualmente sus propios niveles cientos

de veces para verificar que son posibles de completar y para ajustar la dificultad. Eso perjudica gravemente al proyecto al haber mucho tiempo no utilizado de forma eficiente. Jugando, me he dado cuenta de errores como estos al jugar niveles que simplemente no están bien hechos o con bugs (fallos que no deberían estar ahí) que se pueden aprovechar en contra de la empresa.

¿Cómo se resuelve ahora?

La solución actual es **puramente manual**, ya que es el propio programador él cuál **tiene que jugar al nivel repetidamente para probarlo**. Actualmente, se contratan a jugadores o se libera una versión beta para que usuarios reporten errores. Este proceso es **lento, subjetivo (depende de la habilidad del tester) y propenso a errores** humanos causados por la fatiga.

¿Por qué es un problema importante?

Primero, el tiempo que un desarrollador pasa probando un nivel es tiempo que no dedica a crear nuevo contenido, genuinamente está siendo **muy poco productivo y ineficiente**. Por otra parte un nivel **puede parecer imposible** simplemente porque el tester **está cansado**, no porque el diseño esté roto o haya errores, y luego viene la parte del “cuello de botella” en este caso, ya que el lanzamiento de contenido se **puede llegar a retrasar** si no hay suficientes **testers disponibles** para validar los niveles.

¿A quién afecta?

Afecta principalmente a los **estudios Indies** (empresas de desarrollo que están empezando o que no disponen de muchísimos recursos) que no pueden costear grandes equipos de control de calidades, y a los **diseñadores de niveles**, que sufren frustración y pérdida de tiempo en tareas repetitivas

3. Escenario de Uso Real

Para ilustrar la solución, voy a describir el flujo de trabajo de un diseñador de niveles de un juego tipo Geometry Dash.

Situación ACTUAL (Sin la solución):

- 10:00 AM**:** El diseñador **termina de colocar** los obstáculos en un nuevo nivel de alta dificultad.
- 10:15 AM**: Comienza a jugar** el nivel para verificar que todos los saltos son posibles.
- 11:30 AM**:** Tras 50 intentos fallidos en el mismo punto, no sabe si el salto está mal diseñado o si él está jugando mal. **Se frustra**.
- 12:00 PM**:** Decide mover el obstáculo **"por si acaso"**, alterando el diseño original sin certeza técnica.
- Resultado: **Pérdida de 2 horas** de trabajo y un diseño comprometido por la **incertidumbre**.

Situación FUTURA (Con el Agente de Aprendizaje por Refuerzo):

- 10:00 AM: El diseñador termina el nivel. En lugar de jugar, inicia el **sistema de IA**.
- 10:05 AM: El sistema carga el entorno del juego y el agente comienza a **observar el estado y tomar decisiones**.
- 10:10 AM: El diseñador continúa con **otras tareas** mientras el agente **entrena** en segundo plano acelerado.
- 10:30 AM: El sistema notifica que el agente ha encontrado una **ruta exitosa y ha recibido una recompensa positiva**. Se genera un **reporte visua**l de la partida perfecta.
- Resultado: Validación **objetiva en 30 minutos sin esfuerzo humano**. El diseño se mantiene fiel a la visión original.

4. Impacto y Beneficios Esperados

La implementación de este sistema de Aprendizaje por Refuerzo aporta valor cuantificable al proceso de desarrollo

- Ahorro de Tiempo: Automatiza el proceso de **"prueba y error"**, permitiendo realizar miles de iteraciones en minutos.
- Reducción de Costes: Disminuye la **necesidad** de contratar **testers** externos para las fases iniciales de validación.
- Optimización del Gameplay: El agente puede demostrar la **"ruta perfecta"**, sirviendo como **guía** para el **balanceo** del **juego**.
- Mejora de la calidad de vida del desarrollador: Elimina tareas repetitivas y frustrantes.
- Innovación tecnológica: Posiciona al estudio como **pionero** en el uso de **ML** (Machine Learning) aplicado al **control de calidad**.

Aspecto	**Situación Actual**	**Solución con IA**	**Mejora Estimada (%)**
Tiempo de validación de un nivel complejo	4 horas	20 - 30 minutos	-80%
Coste operativo por prueba	Alto ya que necesitas una persona expresamente probando	Bajo ya que solo hay coste de recursos del ordenador	-90%
Objetividad en la detección de fallos	Baja ya que depende de la habilidad de la persona	Alta ya que lo hace mediante algoritmos matemáticos	+100%
Capacidad de testeo 24/7	No ya que la persona trabajará de acuerdo con su jornada	Si ya que es un servicio con una ejecución que no necesita ser supervisada	+200%

5. Viabilidad y Oportunidad de Negocio

El proyecto es técnicamente viable con los recursos disponibles. Se utilizarán tecnologías estándar de la industria y de código abierto:

- Lenguaje: **Python**, ampliamente documentado para **IA**.
- Frameworks: **PyTorch** y **Stable-Baselines3**, que simplifican la implementación de algoritmos de refuerzo.
- Interacción: Librerías como **mss** y **pyautogui** permiten la **captura** de **pantalla** y **control** de **periféricos** sin necesidad de acceder al código fuente del juego comercial, lo que hace la solución versátil.
- Hardware: El entrenamiento de modelos sencillos para juegos **2D** es ejecutable en equipos de **consumo estándar**.

6. Conclusiones y referencias

Después del análisis realizado, realmente pienso que es una muy buena oportunidad de mercado por varias razones, la primera es que la industria del videojuego factura más que el cine y la música juntos. La demanda de herramientas que optimicen el desarrollo está en auge. También es sabido que , aunque existen grandes empresas de IA, pocas ofrecen soluciones específicas para testing de mecánicas de plataformas 2D accesibles para los desarrolladores

Referencias:

Referencias de las tecnologías:

https://gymnasium.farama.org/

https://stable-baselines3.readthedocs.io/

https://pyautogui.readthedocs.io/

Referencias de un reporte que respalda mi idea:

https://modl.ai/state-of-games-qa-report/